

Streaming, SDKs, Runtimes and Hygiene

Track 16.2 – TypeScript for AI Applications · 16-ai-tech-stack/tracks/typescript/notes/02-streaming-runtimes-and-hygiene.md

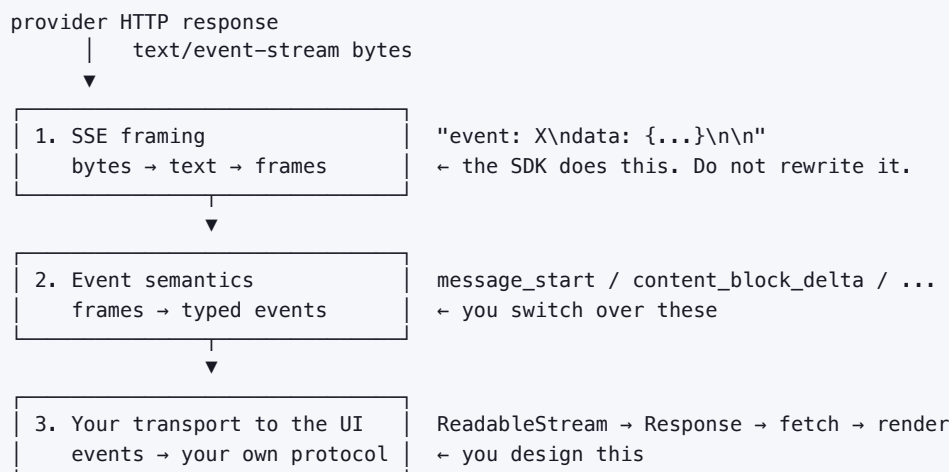
Note 01 was about describing your data. This one is about moving it, running it, and keeping the repo alive after six months.

1. Streaming is a product requirement, not an optimisation

A 600-token answer from a frontier model takes 8–20 seconds to generate. Without streaming your user stares at a spinner for the whole thing and 30% of them leave. With streaming they see the first token in ~500ms and perceive the app as fast. Time-to-first-token is the metric your users actually feel; total latency is the one your dashboard shows.

There is a second, less obvious reason: **streaming is your only cancellation point**. A non-streaming request bills you for every token whether or not the user is still there. A streamed request can be aborted the moment they navigate away.

2. The three layers, and which one you should touch



You should almost never write layer 1. @anthropic-ai/sdk and the AI SDK both hand you a typed async iterator. Write layer 1 only when you are proxying a provider stream through your own edge worker and re-framing it, or debugging someone else's parser. ../code/streaming-parser.ts exists so that when you *do* have to, you know what breaks.

Async iterators are the substrate

```
for await (const event of stream) { /* ... */ }
```

Under the hood: an object with `[Symbol.asyncIterator]()` returning `{ next(), return(), throw() }`. Two things follow that matter enormously:

- **break calls return()**. Breaking out of a `for await` loop tells the producer to clean up. That is how cancellation propagates without any explicit plumbing — and why a `for await` loop you exit with `return` behaves correctly while a manual `.next()` loop leaks.
- **Backpressure is implicit**. The producer does not run ahead of `await`. In a `ReadableStream` the same job is done by `pull` — the runtime calls it only when the consumer's queue has drained. Push everything in `start()` instead and you have reinvented an unbounded buffer.

3. The five bugs, in the order you will hit them

#	Bug	Symptom	Fix
1	Fresh <code>TextDecoder</code> per chunk	São renders as S0000; emoji become 00	<code>one</code> decoder for the stream, <code>decode(chunk, { stream: true })</code>
2	Split on <code>\n\n</code> , drop the remainder	Random missing sentences under load; fine on localhost	keep a buffer across chunks; emit only complete frames
3	No final flush	Last token or last frame silently truncated	<code>decoder.decode()</code> with no args + drain the buffer after the loop
4	<code>error</code> event ignored	Half a sentence, no error anywhere, HTTP 200	handle the <code>error</code> member in your switch; render a visible marker
5	Proxy buffers your stream	Streaming code, non-streaming UX, only in prod	<code>Cache-Control: no-cache, no-transform + X-Accel-Buffering: no</code>

Bugs 1–3 are the same bug wearing three hats: **chunk boundaries are not message boundaries**. TCP knows nothing about SSE frames or UTF-8 code points. Your localhost tests pass because the whole response arrives in one 4KB read; a CDN, a proxy, or a slow mobile link will slice it anywhere.

`streaming-parser.ts` reproduces all of these deterministically by feeding a 7-byte-per-chunk socket to a naive parser, then to a correct one.

Bug 4 deserves more words

A mid-stream failure arrives as an `error event` inside a 200 response whose headers were flushed seconds ago. You cannot change the status code. Your options are: render the partial text and append a visible failure marker, or emit your own `{"type":"error"}` frame on your app-level protocol so the client can show a retry button. What you must not do is let the stream end quietly — the user reads a truncated answer as a complete one.

Rendering partial tokens

Do not `setState` on every token. At 80 tokens/sec that is 80 React renders per second and a janky UI on a mid-range phone. Accumulate into a ref and flush on `requestAnimationFrame`, or batch every ~ 50 ms. The AI SDK's `useChat` already does this; if you hand-roll, do it yourself.

The second rendering trap: **partial Markdown**. A half-streamed response contains an unclosed `` fence or a dangling **``. Use a Markdown renderer that tolerates incomplete input, or hold back the last few characters until a block closes. Rendering raw partial Markdown produces a UI that visibly flickers between "code block" and "not a code block".

4. Cancellation, end to end

```
const ac = new AbortController();
// Browser: abort when the component unmounts or the user clicks Stop.
const res = await fetch("/api/chat", { body, signal: ac.signal });

// Server: the incoming Request carries its own signal when the client hangs up.
export async function POST(req: Request) {
  const stream = client.messages.stream({ ...params }, { signal: req.signal });
  // ...
}
```

The chain is: user clicks Stop → client `AbortController` → HTTP connection closes → server `Request.signal` fires → provider request aborted → billing stops. **Every link must be connected.** Miss the last one and you have a nice Stop button that changes nothing about your invoice.

Treat `AbortError` as normal control flow. Check `err.name === "AbortError"` (or `error instanceof Anthropic.APIUserAbortError` with the Anthropic SDK) and skip the error log — otherwise your alerting fills with users who changed their mind.

5. The SDK layer

Three libraries, three jobs. Picking the wrong one costs you weeks.

Library	What it is	Reach for it when
ai (Vercel AI SDK)	A provider-agnostic façade: <code>generateText</code> , <code>streamText</code> , <code>generateObject</code> , <code>streamObject</code> , <code>tool()</code> , plus React/Svelte/Vue hooks	You are building a chat product, you want provider portability, or you want <code>useChat</code> to handle the client half
@anthropic-ai/sdk	The official Anthropic client. Full API surface, exact types, no abstraction	You need a feature the façade does not expose — prompt caching breakpoints, thinking blocks, server tools, batches, the beta tool runner
@anthropic-ai/claude-agent-sdk	Claude Code packaged as a library: agent loop, built-in file/bash/search tools, subagents, hooks, permissions, MCP	You want a coding or filesystem agent and you do not want to write the loop or the tools

Vercel AI SDK — the product layer

```
import { streamText, tool } from "ai";
import { anthropic } from "@ai-sdk/anthropic";
import { z } from "zod";

const result = streamText({
  model: anthropic("claude-opus-5"),
  messages,
  tools: {
    getWeather: tool({
      description: "Current conditions for a city.",
      inputSchema: z.object({ city: z.string() }),
      execute: async ({ city }) => fetchWeather(city),
    }),
  },
});

return result.toUIMessageStreamResponse();
```

That last line is the reason people use it: it emits a protocol that `useChat()` on the client already understands, including tool-call state. You get streaming, tool round-trips, and optimistic UI without writing a wire format.

`generateObject` / `streamObject` do the structured-output half — you pass a Zod schema and get a validated, typed object back, with `streamObject` giving you a partial object as it fills in.

Version warning. The AI SDK renamed a lot in v5 (`maxTokens` → `maxOutputTokens`, `CoreMessage` → `ModelMessage`, `toDataStreamResponse` → `toUIMessageStreamResponse`, React hooks moved to `@ai-sdk/react`). Blog posts and LLM-generated code will confidently hand you v3/v4 names. Check <https://ai-sdk.dev/docs> before trusting any snippet, including this one.

Anthropic SDK — the escape hatch you will need

```
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic(); // reads ANTHROPIC_API_KEY from the env

const stream = client.messages.stream({
  model: "claude-opus-5",
  max_tokens: 64000,
  messages: [{ role: "user", content: "..."}],
});

stream.on("text", (delta) => process.stdout.write(delta));
const final = await stream.finalMessage(); // full typed Message, usage included
```

Two things to internalise:

- **Use `finalMessage()`.** Do not wrap `.on()` handlers in `new Promise()` — `finalMessage()` already resolves completion, error and abort correctly.
- **Use the SDK's types.** `Anthropic.MessageParam`, `Anthropic.Tool`, `Anthropic.ToolUseBlock`, `Anthropic.ToolResultBlockParam`. Redefining these yourself is the single most common self-inflicted wound in this codebase shape.

For agent loops there is `client.beta.messages.toolRunner({ tools, messages })` with `betaZodTool()` from `@anthropic-ai/sdk/helpers/beta/zod` — you supply the tool functions, the SDK drives the request → execute → resend loop. It is beta; the manual `while (stop_reason === "tool_use")` loop is the stable alternative and is about 30 lines.

Claude Agent SDK — when the agent *is* the product

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Find every TODO in src/ and open one issue summarising them",
  options: { allowedTools: ["Read", "Grep", "Glob"] },
})) {
  console.log(message);
}
```

This is a different product from the two above: it ships the harness *and* the tools (Read/Write/Edit/Bash/Glob/Grep/WebSearch), plus subagents, permission hooks and MCP support. You host it; Anthropic does not run it for you. Docs: <https://code.claude.com/docs/en/agent-sdk> — and the sibling track `../claude-code/` [16-ai-tech-stack/tracks/claude-code].

Do not confuse it with the API SDK's tool runner. The tool runner loops over tools *you* define and has no built-in tools or filesystem access. Same-sounding names, very different scope.

6. Runtimes: where your code actually executes

Runtime	Strength	What bites you
Node 20/22/24	Everything works; the default answer	Cold starts on serverless; ESM/CJS interop
Bun	Fast installs, fast startup, TS out of the box, built-in test runner	Occasional Node-API gaps in less-common libraries; smaller production track record
Deno	Secure-by-default permissions, web-standard APIs, TS native	npm compat is good now but not invisible; smaller ecosystem of guides
Cloudflare Workers	Global, ~0ms cold start, cheap, great for streaming proxies	V8 isolate, not Node. No <code>fs</code> . Node built-ins only behind <code>nodejs_compat</code> . CPU-time caps
Vercel Edge	Same isolate model, integrated with Next.js	Same constraints as Workers; response-duration limits differ by plan

What actually breaks when you move to the edge

1. **No filesystem.** No `fs`, no `path`-based asset loading, no `__dirname`. Bundle your prompts and fixtures as imported modules or fetch them from KV/R2.
2. **Node built-ins are opt-in.** `crypto`, `buffer`, `stream` need the `nodejs_compat` compatibility flag on Workers. Anything reaching for `net`, `dns`, `child_process` or native addons simply will not run.
3. **CPU time is capped, wall time usually is not.** An isolate gets a small CPU budget; waiting on a model's stream is I/O, not CPU, so long generations are normally fine. Heavy client-side work — tokenising, big JSON transforms, image processing — is what trips the limit.
4. **Response duration limits.** Platforms cap how long a single response may stay open. A 10-minute agent run does not belong in a request handler: return a job id, run the work in a queue or durable object, and stream status.
5. **Background work after the response.** Logging, analytics and eval writes must go through `waitUntil()` (or the platform equivalent). A bare floating promise is killed the instant the response finishes.

Opinion: run the chat/streaming route at the edge, where TTFT and global distribution matter most, and keep the agent loop, cron jobs and anything touching a filesystem on Node. Splitting the app this way costs you one extra deploy target and saves you a category of debugging.

7. Project hygiene

tsconfig: the flags that matter

`strict: true` is table stakes. These four are what separate a codebase that catches bugs from one that just has annotations:

Flag	Why
<code>noUncheckedIndexedAccess</code>	<code>arr[0]</code> becomes <code>T undefined</code> . Catches the streaming off-by-one and the empty-content case that every LLM app hits
<code>exactOptionalPropertyTypes</code>	<code>{ a?: string }</code> stops accepting <code>{ a: undefined }</code> . Providers treat missing and null differently; so should your types
<code>verbatimModuleSyntax</code>	Forces <code>import type</code> for type-only imports. Removes a whole class of ESM/CJS runtime surprises
<code>noFallthroughCasesInSwitch</code>	Pairs with <code>assertNever</code> over your stream events

For `module/moduleResolution`: use `"bundler"` for anything running under `tsx/vite/Next`. Use `"nodenext"` only when you are publishing a library that must resolve the way Node does. Setting `"node"` (the TS 4-era default) in 2026 is almost always a mistake — it does not understand `exports` maps.

ESM vs CJS — the tax you pay once

The whole ecosystem has moved to ESM; some of your dependencies have not. Symptoms and fixes:

- `ERR_REQUIRE_ESM` — a CJS file requiring an ESM package. Fix: make your package ESM (`"type": "module"`) or use a dynamic `await import()`.
- `__dirname` is not defined — you are in ESM. Use `path.dirname(fileURLToPath(import.meta.url))`.
- Cannot find module `'./x'` — ESM needs file extensions. Write `./x.js` even from a `.ts` file (yes, `.js`), or use `moduleResolution: "bundler"`.
- Default-import weirdness from a CJS package — `import pkg from "x" giving you { default: ... }`. `esModuleInterop: true` plus `verbatimModuleSyntax` makes this predictable.

Advice: go ESM-only. `"type": "module"` in `package.json`, no dual build. Dual publishing is three days of your life for a shrinking audience.

The rest of the toolchain

Tool	Pick	Why
Package manager	pnpm	Content-addressed store, strict <code>node_modules</code> (a package you did not declare is not importable), fast CI. Workspaces that actually work
Test runner	vitest	Vite-native, ESM-first, watch mode that is genuinely instant, Jest-compatible API. Use snapshot tests sparingly on LLM output — they are the definition of brittle
Lint/format	biome	One Rust binary replacing eslint + prettier, ~10× faster, near-zero config. Stay on eslint only if you need a plugin biome lacks
Type checking in CI	<code>tsc --noEmit</code>	<code>tsx/esbuild/swc</code> do not typecheck — they strip types and run. If <code>tsc</code> is not in CI, you do not have types, you have comments

That last row is the one people get wrong. `npx tsx app.ts` will happily run code with a dozen type errors in it.

Testing LLM code without a network

Three levels, in order of value per hour:

- 1. **Pure functions** — parsers, schema validators, prompt builders. Fast, deterministic, and where most of your real bugs live. Both scripts in `../code/` are written to be testable this way.
- 2. **Recorded fixtures** — capture one real SSE response to a file and replay it through your parser at hostile chunk sizes. This is the highest-value test in an AI codebase and almost nobody writes it.
- 3. **Live eval** — against the real model, on a schedule, not in CI. See `../15-ai-evals/` [15-ai-evals].

Never mock the model's *content* and call it a test of your prompt. That tests your mock.

8. Where this returns

Idea here	Where it comes back
SSE framing + the event switch	<code>../06-ai-agents/</code> [06-ai-agents] — streaming an agent's intermediate steps
<code>AbortController</code> through every layer	<code>../10-ai-architecture/</code> [10-ai-architecture] — timeouts, budgets and backpressure
Zod tool schemas over a transport	<code>../09-mcp/</code> [09-mcp] — MCP's TS SDK takes Zod directly
Recorded-fixture tests	<code>../15-ai-evals/</code> [15-ai-evals] — regression suites for prompts
Edge limits and job queues	<code>../aws-bedrock/</code> [16-ai-tech-stack/tracks/aws-bedrock], <code>../langgraph/</code> [16-ai-tech-stack/tracks/langgraph] — long-running orchestration
Claude Agent SDK	<code>../claude-code/</code> [16-ai-tech-stack/tracks/claude-code]